

GroundedNutriRec Technical Report: Week 2

Preprocessing and Feature Engineering

Framework: GroundedNutriRec: Retrieval-Augmented Multi-Objective LLM Framework for Health-Aware and Explainable Food Recommendation

Report Type: Academic Technical Paper (Week 2 Review)

Authors: GroundedNutriRec Core Development Team

1. Introduction

Following the exploration of the raw Food.com dataset, the second week of the GroundedNutriRec summer project focuses on establishing two parallel, reproducible data pipelines:

1. Interaction Preprocessing Pipeline (Notebook 03 — Krina Parikh): Cleans and restructures the Food.com user-recipe interaction log (`RAW_interactions.csv`) into a form suitable for collaborative filtering baseline models. This includes duplicate removal, zero-rating identification, user-mean normalization, iterative K-Core filtering, and implicit binary label derivation.

2. Recipe Feature Engineering Pipeline (Notebook 04 — Ishan Shastri): Augments the cleaned recipe metadata file (`Food_metadata_clean.csv`) with derived nutritional and preparation complexity features to support health-aware ranking and multi-objective recommendation objectives.

This report documents the design decisions, mathematical formulations, and empirical outputs produced by both notebooks.

2. Interaction Data Preprocessing (Notebook 03)

The raw Food.com interaction dataset (`RAW_interactions.csv`) was loaded and preprocessed using the following sequential pipeline.

2.1 Dataset Schema and Initial Statistics

The raw dataset contains the following columns:

Column	Description
user_id	Unique identifier for each Food.com user
recipe_id	Unique identifier for each recipe
date	Date of the interaction (review or rating)
rating	Explicit rating on a 0–5 scale (0 = review without rating)
review	Free-text review content

Initial dataset shape: **1,132,367 rows × 5 columns**. No null values were present in any column except review (169 missing), which is not used for model training.

2.2 Step 1 — Duplicate Interaction Removal

A duplicate interaction is defined as any case where a single user rated the same recipe more than once. For collaborative filtering, each user-item pair must appear at most once. The pipeline checks for duplicate (user_id, recipe_id) pairs and retains only the most recent interaction by dropping duplicates.

Result: **No duplicate interactions were found.** The dataset retained its shape of 1,132,367 rows.

2.3 Step 2 — Zero-Rating Identification

In the Food.com dataset, a rating value of 0 indicates that a user left a written review but did not assign an explicit star rating. These are treated as implicit interactions — the user demonstrated engagement with the recipe, but provided no signal of preference direction.

Zero-rating count: 60,847 interactions (5.37% of the dataset)

Null ratings: 0

These records are retained but flagged separately from rated interactions in the implicit feedback derivation step.

2.4 Step 3 — User-Mean Rating Normalization

To compensate for individual user rating biases (some users habitually rate all recipes high; others rate consistently low), user-mean centering is applied to all explicit (non-zero) ratings:

$$\text{normalized_rating} = \text{rating} - \text{user_mean_rating}$$

where *user_mean_rating* is the mean of all explicit ratings by that user across the full dataset.

Global explicit rating mean: 4.6615

The normalization produces centered ratings (positive = above user's average preference, negative = below). Zero-rated interactions are assigned NaN for the normalized rating field and are excluded from mean computation.

Sample output:

user_id	recipe_id	rating	user_mean_rating	normalized_rating
38094	40893	4	4.828571	-0.828571
1293707	40893	5	4.913043	+0.086957
8937	44394	4	4.225806	-0.225806
126440	85009	5	4.848016	+0.151984
57222	85009	5	4.548837	+0.451163

2.5 Step 4 — Iterative K-Core Filtering

Collaborative filtering suffers from extreme sparsity and cold-start problems when applied to users with very few interactions or recipes with very few ratings. K-Core filtering addresses this by iteratively pruning all users with fewer than K interactions and all recipes with fewer than K ratings until the remaining dataset converges to a stable subgraph.

Configuration: $K = 5$

The algorithm converged after **10 iterations**:

Iteration	Remaining Rows
Start	1,132,367
1	611,544
2	565,002
3	557,457
4	555,986
5	555,706
6	555,654
7	555,634
8	555,626
9	555,622
10 (converged)	555,618

Final K-Core filtered dataset statistics:

Total interactions: 555,618

Unique users: 17,813

Unique recipes: 41,240

Interaction matrix sparsity: 99.9244%

2.6 Step 5 — Implicit Feedback Label Derivation

Explicit ratings on the 0–5 scale are converted into binary implicit feedback labels to simplify recommendation model training:

Condition	Label
rating $\geq$ 4	liked = 1 (positive preference signal)
0 < rating < 4	liked = 0 (negative preference signal)
rating = 0	liked = NaN (no explicit preference, implicit engagement only)

Distribution of implicit feedback labels in the 5-core filtered dataset:

Label	Count
liked = 1	517,549
liked = 0	24,997
liked = NaN	13,072

Sample output showing the final preprocessed schema:

user_id	recipe_id	date	rating	user_mean_rating	normalized_rating	liked
202555	225241	2007-06-20	5	4.731707	+0.268293	1.0
353579	225241	2007-08-14	5	4.771987	+0.228013	1.0
681408	225241	2008-03-14	0	5.000000	NaN	NaN
684460	225241	2009-01-18	5	5.000000	0.000000	1.0
900992	225241	2009-02-19	3	3.857482	-0.857482	0.0

The preprocessed interactions dataset is saved as `interactions_clean.csv` for use in baseline recommender training.

3. Recipe Feature Engineering (Notebook 04)

The recipe metadata dataset (`food_metadata_clean.csv`) contains per-recipe nutritional and preparation attributes. Six derived features were engineered to support health-aware recommendation objectives.

3.1 Dataset Schema

The raw metadata file contains the following columns:

Column	Description
<code>item_id</code>	Unique recipe identifier
<code>recipe_name</code>	Human-readable recipe title
<code>ingredients</code>	Comma-separated ingredient list
<code>calories</code>	Total caloric content per serving (kcal)
<code>protein</code>	Protein content per serving (grams)
<code>fat</code>	Total fat content per serving (grams)
<code>carbs</code>	Total carbohydrate content per serving (grams)
<code>category</code>	Recipe category label
<code>prep_time</code>	Total preparation and cooking time (minutes)
<code>instructions</code>	Step-by-step preparation instructions

Dataset shape: 522,517 rows × 10 columns

All numeric columns (`calories`, `protein`, `fat`, `carbs`, `prep_time`) were coerced to numeric type with zero-filling for any non-parseable values.

3.2 Engineered Features

Six features were derived from the raw metadata fields:

3.2.1 Calorie Level (`calorie_level`)

A categorical classification of recipe caloric density into three tiers:

Level	Condition
Low	calories < 200 kcal
Medium	$200 \leq \text{calories} < 500 \text{ kcal}$
High	calories $\geq 500 \text{ kcal}$

3.2.2 Protein Level (protein_level)

A categorical classification of protein content per serving:

Level	Condition
Low	protein $\leq 10\text{g}$
Medium	$10\text{g} < \text{protein} \leq 20\text{g}$
High	protein > 20g

3.2.3 Fat Level (fat_level)

A categorical classification of total fat content per serving:

Level	Condition
Low	fat $\leq 10\text{g}$
Medium	$10\text{g} < \text{fat} \leq 20\text{g}$
High	fat > 20g

3.2.4 Ingredient Count (ingredient_count)

The number of distinct ingredients in each recipe, computed by counting comma-separated tokens in the ingredients field. This serves as a proxy for recipe complexity at the ingredient composition level.

3.2.5 Health Score (health_score)

A continuous health score is computed for each recipe using the following formula:

$$\text{health_score} = (\text{protein} \times 2) - \text{fat} - (\text{calories} \times 0.01)$$

This formula rewards high-protein recipes and penalizes high-fat and high-calorie recipes, producing a score that increases monotonically with nutritional quality relative to these three macronutrient dimensions.

3.2.6 Preparation Complexity (preparation_complexity)

A three-level categorical classification based on preparation time:

Level	Condition
Easy	prep_time < 30 minutes
Medium	30 ≤ prep_time < 60 minutes
Hard	prep_time ≥ 60 minutes

3.3 Feature Engineering Output Sample

The following table shows the engineered feature values for the first five recipes in the dataset:

calories	calorie_level	protein	protein_level	fat	fat_level	ingredient_count	health_score	prep_time	preparation_complexity
170.9	Low	3.2	Low	2.5	Low	4	2.191	0.0	Easy
1110.7	High	63.4	High	58.8	High	25	56.893	0.0	Easy
311.1	Medium	0.3	Low	0.2	Low	7	-2.711	0.0	Easy
536.1	High	29.3	High	24.0	High	14	29.239	0.0	Easy
103.6	Low	4.3	Low	0.4	Low	5	7.164	0.0	Easy

The full engineered feature set is saved as `food_features_engineered.csv` for use in health-aware ranking and multi-objective scoring pipelines.

4. Summary of Week 2 Outputs

Artifact	Description	Produced by
interactions_clean.csv	555,618 preprocessed user-recipe interactions with normalized ratings and implicit labels	Notebook 03 (Krina Parikh)
food_features_engineered.csv	522,517 recipes augmented with calorie level, protein level, fat level, ingredient count, health score, and preparation complexity	Notebook 04 (Ishan Shastri)

5. Description of K-Core Sparse Filtering

5.1 Brief Introduction

K-Core filtering is an iterative graph-cleaning algorithm used to prune sparse nodes from bipartite user-item interaction graphs, retaining only a core subset of active users and frequently-interacted items.

5.2 Detailed Explanation

The algorithm takes an interaction table and filters out users with fewer than K interactions and items with fewer than K ratings. Because removing a user reduces the interaction counts of their associated items, item degrees can fall below the threshold. The algorithm therefore alternates iteratively between user-level and item-level pruning until convergence.

5.3 Examples

In our preprocessing pipeline, a 5-core filter is applied to the Food.com interaction log. An interaction between user U and recipe R is retained only if user U has at least 5 interactions remaining in the corpus and recipe R has been rated by at least 5 active users. The algorithm required 10 iterations to converge, reducing 1,132,367 interactions to 555,618.

5.4 Advantages

Reduces Sparsity: Reduced the Food.com sparsity to 99.9244%, making collaborative filtering algorithms computationally stable.

Reduces Cold-Start Noise: Removes users with negligible histories, ensuring that evaluation metrics (HitRate@K, NDCG@K) are computed on profiles with sufficient history.

5.5 Disadvantages

Data Reduction: Prunes a substantial fraction of interactions, discarding information about rare items and inactive users.

Optimism Bias: Models evaluated on K-Core filtered datasets may exhibit artificially inflated accuracy metrics relative to real-world deployment.

5.6 Use Cases

Collaborative filtering model training and evaluation.

Sequential recommender training where minimum sequence length per user is required.

5.7 Limitations

Cannot be used to evaluate cold-start recommendation strategies, as it actively filters out cold-start profiles.

6. Description of the Health Score Formula

6.1 Brief Introduction

The health score is a continuous, formula-based metric that evaluates the relative healthiness of a recipe by combining three macronutrient signals: protein content (positive), fat content (negative), and caloric density (negative).

6.2 Detailed Explanation

The formula $\text{health_score} = (\text{protein} \times 2) - \text{fat} - (\text{calories} \times 0.01)$ assigns a positive reward to protein (which supports satiety and muscle maintenance) while penalizing total fat content and overall caloric density. The coefficient 2 on protein doubles its contribution relative to fat, reflecting dietary guidance that prioritizes protein adequacy. The 0.01 scaling factor on calories ensures that caloric contribution is proportionally weighted without dominating the score.

6.3 Examples

A recipe with protein = 30g, fat = 10g, and calories = 400 kcal receives:

$$\text{health_score} = (30 \times 2) - 10 - (400 \times 0.01) = 60 - 10 - 4 = \mathbf{46.0}$$

A recipe with protein = 5g, fat = 30g, and calories = 800 kcal receives:

$$\text{health_score} = (5 \times 2) - 30 - (800 \times 0.01) = 10 - 30 - 8 = \mathbf{-28.0}$$

6.4 Advantages

Simple and Interpretable: Formula is straightforward to audit and communicate.

Continuous and Differentiable: Produces a smooth gradient suitable for multi-objective ranking models.

6.5 Disadvantages

Does Not Account for Daily Reference Values: Does not normalize against WHO or FSA dietary reference values.

Ignores Micronutrients and Carbohydrates: Focuses on three macronutrients, omitting carbohydrates, vitamins, minerals, and fiber.

6.6 Use Cases

Multi-objective recommendation ranking combining user preference with nutritional quality.

User profile health alignment and filtering.

6.7 Limitations

Score may be negative for high-calorie or high-fat recipes, which can complicate direct comparison without normalization.

Does not distinguish between types of fat (saturated vs. unsaturated) or types of protein.